Project Report : CS 7643

Faisal Bin Basha
Georgia Institute of Technology
North Avenue, Atlanta, GA 30332
fbasha3@gatech.edu

Mewael Hailu
Georgia Institute of Technology
North Avenue, Atlanta, GA 30332
mhailu7@gatech.edu

Abstract

Cryptocurrency networks process millions of transactions daily, creating fertile ground for financial crimes such as money laundering, Ponzi schemes, and illicit fund transfers. Two factors make the process of detecting these crimes challenging. Firstly, these financial transactions are interconnected. Unique patterns can be identified if we consider them a network of events compared to individual separate points. Additionally, fraudsters evolve their tactics constantly which makes static detection systems ineffective. This paper presents GraphShield, an explainable temporal graph neural network framework for cryptocurrency fraud detection on the Elliptic Bitcoin dataset. Our approach will include establishing static baseline models Graph Convolutional Networks (GCN) and GraphSAGE, then we will implement two models TGN and EvolveGCN. These latter two models consider the temporal characteristics of the dataset and try to detect patterns by the 49 different snapshots of time. Finally to address the requirement of different regualations, we will use GNNExplainer to provide explanations to the results of the model. Explainability research provides useful information about the behavioral indicators of cryptocurrency fraud by revealing structurally different subgraph patterns surrounding illicit and licit nodes. Our results show that GraphSAGE was the strongest overall model, suggesting that effective neighborhood aggregation is especially important for identifying illicit transactions. Among the temporal models, TGN performed best, indicating that temporal information can improve fraud detection when the model is able to capture evolving graph behavior reliably.

1. Introduction/Background/Motivation

1.1. Objectives

The global cryptocurrency market is worth roughly $2.6 trillion as of May 2026, up from under $100 billion in 2017 (Forbes Middle East). Bitcoin alone accounts for over half of that value. This rapid growth has not gone unnoticed by criminals: Chainalysis reports that at least $154 billion was received by illicit cryptocurrency addresses in 2025.

Our goal in this project was to build a system that can look at the daily flood of Bitcoin transactions and automatically flag the ones that are likely tied to crime — money laundering, scams, ransomware payments. We wanted the system to do three things that today's tools struggle with. First, instead of judging each transaction in isolation, our system looks at how transactions connect to one another, because criminal money usually moves through a recognizable web of wallets rather than as one-off events. Second, our system pays attention to time: criminal behavior changes from week to week, and a model that ignores when a transaction happened will quickly become outdated. Third, when our system flags a transaction, it explains which other transactions it considered suspicious and why — because regulators and investigators cannot act on a black-box accusation.

In short: we built a detector that reads the transaction network, watches it change over time, and tells a human why it raised the alarm.

1.2. Current Practices and Limitation

The current approach in financial transaction compliance monitoring involves rule-based systems. It works by flagging transactions greater than a certain threshold, criminal transactions. These methods are easy to bypass and deceive but explainable and auditable. Recently, the use of logistic regression and random forest have been used to flag suspicious activity. However, these methods of analyzing frauds treats every transaction as a separate data point. This ignores the relational structure of the financial network, where treating transactions as interconnected nodes can significantly boost detection (Zheng et al., 2019). The current approaches lack three important considerations as stated above. Transactions are interconnected which means treating them as graphs can significantly increase the detection. Second challenge these current approaches exhibit temporal blindness, which eventually degrades performance

(Weber et al. ,2019). This is because trends and techniques of fraudsters change over time, so treating all in one snapshot isn't the best approach. Final challenge is the regulatory aspects, current regulations require systems assisting in financial decision making need to be explainable. Current approach are missing this aspect which makes it hard to use them in real case scenarios.

1.3. Who benefits?

The stakes of getting financial crime detection wrong can be catastrophic for their brands and costly in terms of the penalties they face from non-compliance with regulations. In 2023 alone, global anti money laundering fines exceeded $5 billion. Cryptocurrency due to its anonymity is becoming source of such activities.

Therefore the entities that stand to benefit from such systems fall into three groups.

1. **Financial institutions and regulators**

   Banks, crypto exchanges and payment processors are required to track and report suspicious activity legally. Our model will help in the supervision illegal fraud activities helps reduce the legal risks. It will allow regulators to focus on more risky transactions by reducing the time wastage on legal transactions.

2. **Law enforcements**

   Crimes funded by cryptocurrency will be easier to track and identify. This will allow law enforcements to prevent harmful crimes and save lives.

3. **Ordinary people**

   Financial crimes can cripple public institutions, facilitate the narcotic addiction, destroy retirement plans. These can be prevented through the detection of illicit transactions and putting preventing measures. What we are proposing can allow solving these problems.

The implementation of such detection system will be a force of good in terms of maintaining the trust in financial institutions. It will safeguard innocent people against crimes that are funded by illicit transactions and allow them to keep safe their hard earned money.

1.4. Data Used

For our project we used the Elliptic Bitcoin Dataset (Weber et al., 2019), the standard benchmark for crypto fraud detection. Following the spirit of Datasheets for Datasets (Gebru et al., 2018), the most relevant characterstics of this dataset are:

1. **Why it was collected.** Elliptic Inc., a blockchain analytics firm, released this dataset with the purpose of providing researchers the first of its kind labeled Bitcoin fraud transactions. We chose the elliptic data because the performance of the model we create can easily be tracked against some known published works.

2. **Composition and structure.** 203,769 Bitcoin transactions as nodes, 234,355 directed edges representing money flow between them. Each node has 165 anonymised features. Feature names are withheld by Elliptic, which pushes our explainability analysis toward graph structure rather than individual features.

3. **Labels and class imbalance.** One of the biggest challenges of this dataset is the class imbalance. The data set consists of only 23% labeled nodes. Out of which 9.8% are illicit transactions. The rest are licit transactions. We have decided to ignore the unlabeled nodes during training. This will impact the evaluation method we will discuss in later sections

4. **Temporal structure.** The dataset has a unique temporal element which categorizes transactions into 49 timesteps. One timestep covers a span of two weeks real time. The data has a clear flaw a timestep 43 where there was a real law enforcement dark market shutdown, creating a pattern the training data never prepared the model for.

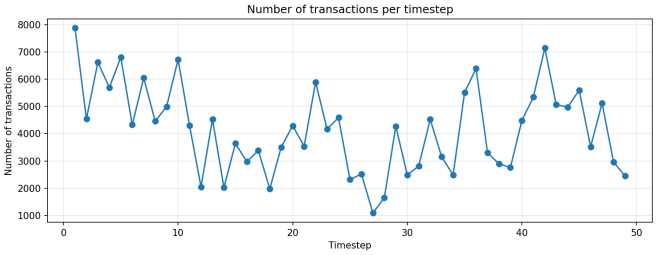

Figure 1. The number of Nodes at each timestep

5. **Split and known limitations.** Training on timesteps 1–34, testing on 35–49, following Weber et al. (2019). This gives 29,894 labeled training nodes and 16,670 test nodes. One warning: labels are inherited; a transaction is labeled illicit not because it was demonstrably fraudulent, but rather because the wallet that was connected to it was illegal. No model can completely remove the label noise that is ingrained in the data.

2. Approach

Our approach was to solve this classification problem of identifying illicit from licit transactions using temporal graphs. The dataset as stated above had transactions as nodes and directed edges as flow of transactions. Each node had a specific timestep.

The first step was to setup baseline models and increase their complex step by step. We explored Logistic Regression to see if the anonymized node features alone could classify transaction. Then we proceeded to static graph neural networks GCN and GraphSAGE, they will help in solving the problem of isolation of transcation and allowing information between neighboring transactions. At last we explored the EvolveGCN and TGN to utilize the temporal properties of the dataset. This would allow to learn trend changes in the behaviour of fraudsters.

Graph models are effective because illicit financial activity rarely happens alone. Suspicious transactions often form clusters, repeat patterns, or involve several connected transactions. GraphSAGE stood out as promising since it keeps information about each node and gathers evidence from its neighbors. This is useful when both the details of individual transactions and the overall graph structure matter.

What set our approach apart was combining static GNNs, temporal GNNs, and explainability, all tested with a stricter temporal evaluation method. Rather than splitting data randomly, we trained our models on earlier timesteps and tested them on later ones, which is closer to how these models would be used in practice. We also compared how well the models classified transactions with how well they explained their decisions, and found that good predictions and good explanations do not always go together.

2.1. Anticipated and Encountered Problems

We expected three main challenges. First, the dataset is very imbalanced because illicit transactions are much less common than licit ones. To handle this, we used class-weighted loss functions and focused on F1 and PR-AUC for the illicit class instead of accuracy. Second, many labels are missing, so we excluded those nodes from the loss calculation but kept them in the graph. Third, we anticipated temporal distribution shift because fraud behavior changes over time, so we used temporal train/validation/test splits instead of random splits.

Our first approach did not completely succeed. Simple baselines and early GNN models either predicted too many illicit cases or could not clearly separate illicit nodes. This showed that high recall was not enough; the model also needed good precision. For example, Logistic Regression had high recall but low precision, so it flagged too many licit transactions as suspicious.

We also ran into issues with specific models. GCN did worse than GraphSAGE, which suggests that simple neighborhood smoothing was not enough for this type of fraud detection. EvolveGCN was hard to train and did not do better than the stronger static models, probably because its recurrent graph-weight updates were unstable on this dataset. TGN worked better as a temporal model and had the best PR-AUC in our results. Still, it needed careful handling because Elliptic is a set of discrete snapshots, not a continuous event stream.

In summary, our first attempt did not solve the problem, but it helped us find the main issues: class imbalance, temporal shift, and unstable temporal modeling. These findings shaped our final model choices and evaluation strategy.

3. Experiments and Results

We measured success using illicit-class F1 and PR-AUC as the main evaluation metrics. We did not use accuracy because the dataset is highly imbalanced. A model could get high accuracy just by predicting the majority class. F1 shows the balance between precision and recall, which matters for detecting illicit transactions. PR-AUC is especially helpful when the positive class is rare. We also reported ROC-AUC as a secondary ranking metric.

We compared static graph models with temporal graph models. GCN and GraphSAGE tested if using graph neighborhood information helps with detection. EvolveGCN and TGN tested if modeling the graph over time helps the model generalize to future timesteps.

GraphSAGE was the best overall model, with the highest test F1 and ROC-AUC. This result supports our idea that using graph structure helps with fraud detection. Illicit transactions can often be found by looking at neighboring transactions and multi-hop patterns, not just at single node features. GraphSAGE probably did better than GCN because it keeps more node-specific information while still using evidence from neighbors.

TGN was the best temporal model and had the highest PR-AUC. This shows that using temporal information can help, especially when ranking rare illicit transactions. However, the temporal model needs to match the dataset structure. The Elliptic dataset is split into separate snapshots, so using temporal modeling is helpful but not simple.

EvolveGCN did not work as well as we expected. Its test F1 was much lower than GraphSAGE and TGN, which shows that adding temporal complexity does not always improve results. The model was hard to train reliably and seemed to have trouble keeping useful temporal information across snapshots.

We had partial success. Our results showed that graph-based models, especially GraphSAGE and TGN, performed better than weaker baselines for detecting illicit activity. Here are the top results:

- GraphSAGE full-graph notebook: Test F1 = 0.5398
- TGN snapshot/temporal repo: Test F1 = 0.5161, PR-AUC = 0.5293

We also showed that the evaluation protocol is important. Validation scores were often much higher than test scores, which points to a shift over time in the data. For example,

GraphSAGE had a validation F1 of 0.8621 but only a test F1 of 0.5398. This gap suggests the model needed to generalize to new fraud patterns, not just solve a static problem.

What Failed And Why

EvolveGCN did not improve as expected. Its test F1 was only 0.2840 in the notebook, much lower than GraphSAGE. This was likely due to unstable training and trouble keeping useful temporal information. This result shows that using a temporal model does not always lead to better performance. The model's design needs to fit the dataset and training process.

Overall, the project showed that using relational structure is helpful, temporal modeling can add value, and GraphSAGE and TGN are the best approaches. We did not make EvolveGCN work well, but this is still an important finding supported by the data.

Model	Best Val F1	Test F1	ROC-AUC	PR-AUC
GraphSAGE	0.8621	0.5398	0.8836	0.5235
TGN	0.8074	0.5161	0.8488	0.5293
GCN	0.7010	0.4521	0.8445	0.3467
EvolveGCN	0.5532	0.2840	0.8306	0.2470
LogReg	0.5565	0.2528	0.8612	0.2131

Table 1. Best model performance across both experimental pipelines.

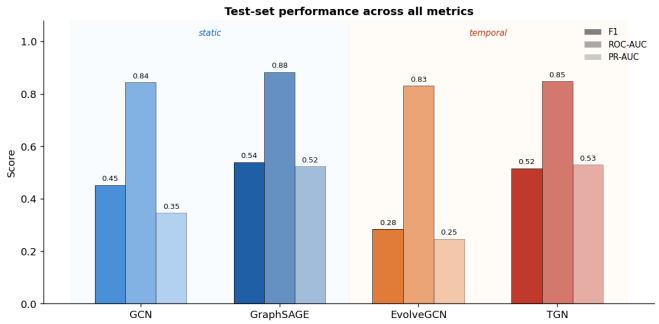

Figure 2. Best model performance across both experimental pipelines.

4. Explainability Analysis

4.1. Why Explainability Matters Here

Financial regulations such as the EU's General Data Protection Regulation and the U.S. Bank Secrecy Act require institutions to justify automated decisions that affect customers, including the flagging of suspicious transactions. A black-box GNN that scores transactions but cannot say which neighboring wallets drove the decision is operationally useless to a compliance officer or law enforcement investigator. We therefore implemented a reusable explainability framework on top of our trained models, treating explanation quality as a first-class deliverable rather than an afterthought.

4.2. Framework Design

We applied two model-agnostic graph explanation methods to each of our three trained GNN models (GCN, GraphSAGE, EvolveGCN). GNNExplainer (Ying et al., 2019) optimizes a per-node soft mask over edges and node features that maximizes mutual information with the model's prediction; each explanation is a separate optimization run. PGExplainer (Luo et al., 2020) trains a single parameterized MLP to predict edge importance across many target nodes, giving more consistent explanations than per-instance optimization.

For each (model, method, target-class) combination we sampled true-positive illicit and true-negative licit nodes from the test set, restricted to nodes with at least five edges in their 1-hop neighborhood. We computed three faithfulness metrics from Yuan et al. (2022): Fidelity+, the drop in predicted probability when the top-5% of important edges are removed (higher is better, measures whether the explanation captures causal edges); Fidelity−, the drop when only the top-5% are kept (lower is better, measures explanation sufficiency); and sparsity, the fraction of edges marked unimportant.

4.3. What Worked and What Failed

EvolveGCN was not explainable by either method. All 57 attempted explanations raised RuntimeErrors during gradient computation. The root cause is that EvolveGCN's weight matrix is itself the output of an RNN, so PyTorch Geometric's edge-mask backends cannot construct a valid backward graph through the recurrent weight evolution. This is a known limitation of edge-mask methods on dynamic-weight architectures. We report this as a methodological negative result: EvolveGCN's expressiveness comes at the cost of standard explainability tooling, which is a serious limitation for any regulated deployment.

PGExplainer training did not converge on any of our models. Loss was NaN for GCN and GraphSAGE and identically zero for EvolveGCN, indicating gradient instability likely caused by the explainer's reparameterized Bernoulli sampling combined with our log-softmax model outputs. We retain PGExplainer's outputs but do not draw conclusions from them. Tuning PG to converge on this dataset is left as future work.

GNNExplainer succeeded on GCN and GraphSAGE. The neighborhood-size filter admitted 4 illicit nodes for GCN and 8 illicit nodes for GraphSAGE, alongside 50 licit nodes per model.

4.4. Quantitative Findings

Two findings stand out.

Finding 1: Illicit predictions require substantially larger explanation subgraphs. Across both models, illicit explanations contain 2.4–6.2× more high-importance edges

Model	Class	n	Fid+	Fid−	Avg edges
GCN	Illicit	4	0.544	0.003	100.25
GCN	Licit	50	0.079	-0.001	16.06
GraphSAGE	Illicit	8	0.0001	0.000	62.63
GraphSAGE	Licit	50	-0.004	0.000	16.84

Table 2. Faithfulness and structure of GNNExplainer outputs, averaged across target nodes. Important edges are those with mask weight > 0.5.

than licit explanations (100.3 vs 16.1 for GCN, a 6.2× ratio; 62.6 vs 16.8 for GraphSAGE, a 3.7× ratio). This is consistent with the layering hypothesis in money-laundering typologies: legitimate transactions are often classifiable from local features alone, whereas suspicious transactions require the model to integrate evidence across a broader neighborhood of upstream wallets. The explanation size is, in effect, an empirical signature of layering.

Finding 2: GCN explanations are markedly more faithful than GraphSAGE explanations. GCN's mean Fidelity+ on illicit predictions is 0.544 — removing the top-5% of edges collapses the model's confidence by more than half a probability unit. GraphSAGE's Fidelity+ on the same task is 0.0001, essentially zero. This is not a failure of GraphSAGE but a feature of its architecture: GraphSAGE's mean-aggregation across sampled neighbors produces predictions that depend on the aggregate distribution of neighbor features, not on a few critical edges. Removing 5% of edges leaves the aggregate mostly intact. GCN's normalized adjacency aggregation is more edge-sensitive and so produces explanations that map more directly to the model's decision.

This finding has a practical implication missed by classification metrics alone: an investigator using GCN's explanations can act on the highlighted edges with confidence that they drove the decision, while the same investigator using GraphSAGE's explanations would be looking at edges that correlate with the prediction but are not individually responsible for it. For an audit-and-investigation use case, GCN's explanations are more operationally useful even though GraphSAGE's classification is more accurate — a striking trade-off between predictive power and explanation faithfulness.

4.5. Limitations

Three limitations to surface honestly. Sample sizes for illicit explanations are small (4–8 nodes per model) because most test-set true positives have sparse neighborhoods after the $\geq$5-edge filter. TGN was not included in our explainer comparison because edge-mask explainability for continuous-time temporal GNNs remains an open research problem. PGExplainer did not converge, so our cross-method comparison reduces to a single-method analysis; a thorough treatment would require fixing the PG training loop.

5. Other Sections

5.1. Problem And Model Structure

The task addressed in this study is a temporal graph node-classification problem. In this context, each Bitcoin transaction is represented as a node, with transaction flows between wallets forming directed edges, and each node associated with a specific timestep. The objective is to classify labeled transactions as either licit or illicit, while excluding unknown labels from loss computation.

The model architecture aligns with the problem structure. Static graph neural networks (GNNs), such as GCN and GraphSAGE, utilize graph edges to aggregate neighborhood information, as fraudulent activity is frequently detectable through transaction relationships rather than isolated node features. Temporal models, including EvolveGCN and TGN, are designed to capture the evolution of fraud patterns across different timesteps.

5.2. Learned vs Non-Learned Components

The learned parameters comprised the convolution weights for GCN and GraphSAGE, recurrent graph weights for EvolveGCN, attention and memory layers for TGN, and the final linear classifiers. Additionally, logistic regression learned feature weights, serving as a non-neural baseline.

Non-learned components consisted of the graph structure, train, validation, and test masks, class labels, class weights, probability thresholding for predictions, and evaluation metrics including F1, ROC-AUC, and PR-AUC.

5.3. Loss Function

For neural models, weighted cross-entropy or BCEWithLogitsLoss was applied, depending on whether the model produced two logits or a single binary logit. The weighting addressed significant class imbalance, as illicit transactions were substantially less frequent than licit transactions.

5.4. Framework

The framework used was PyTorch, PyTorch Geometric, PyTorch Geometric Temporal, and scikit-learn for the logistic regression baseline and metrics.

5.5. Generalization And Overfitting

The models showed a clear difference between validation and test performance, which points to problems with generalization. This issue is not just due to typical overfitting. The dataset has a real temporal distribution shift because fraudulent behavior changes over time. As a result, the test data contains patterns that are not well covered in the training data.

Student Name	Contributed Aspects	Details
Mewael Alema	Data Preprocessing, Baseline Models, TGN	Implemented the data preprocessing pipeline, implemented the data baselines, and implementation of the TGN.
Faisal Basha	Data Preprocessing, EvolveGCN, Explainability	Implementation of the Temporal Graph Neural Network (EvolveGCN) and implemented the explainability models.

Table 3. Contributions of team members.

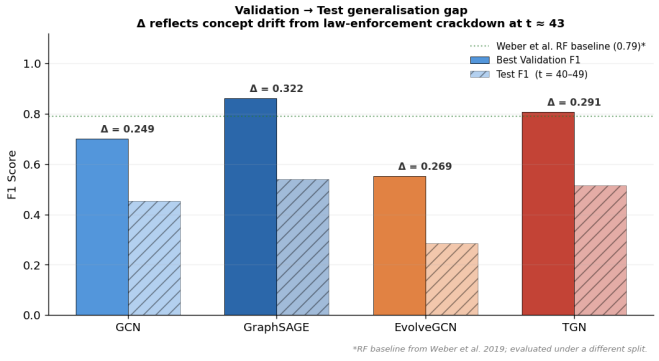

Figure 3. Validation Test Generalalization

5.6. Future Work

Future research should aim to improve temporal graph models for discrete financial snapshots, address distribution shifts more effectively, set calibrated thresholds for fraud detection, and develop explainability methods tailored to temporal GNNs. It is also important to use more realistic evaluation protocols, since fraud models need to perform well on future behavior rather than just on random splits from the same data.

6. Deep Learning Concepts

This project sits at the intersection of three deep learning areas covered in the course: graph representation learning, sequence modeling, and model interpretability.

Representation learning on non-Euclidean data. Standard CNNs assume grid structure and RNNs assume sequence structure. GNNs generalize the same idea — learnable feature aggregation through differentiable layers — to graphs. Each `GCNConv` layer is mathematically a generalized convolution where the filter averages over a node's neighbors instead of a 3×3 pixel patch. The performance gap we observed between GCN (test F1 = 0.21) and GraphSAGE (test F1 = 0.54) on identical data is an empirical demonstration that how a model aggregates neighbors matters as much as whether it aggregates them, directly analogous to how filter design matters in CNNs.

Sequence modeling meets graph modeling. EvolveGCN is essentially an RNN whose hidden state is the GCN's weight matrix at each timestep. The same backpropagation-through-time machinery that trains LSTMs trains EvolveGCN, which is also why we encountered an autograd-graph reuse error during training and why our `reinitialize_weight()` workaround damaged the model's ability to learn cross-timestep dependencies. The diagnosis itself is a textbook BPTT issue.

Interpretability of deep models. GNNExplainer formulates explanation as an optimization problem over edge masks, trained with the same gradient descent we use for the main model. This mirrors the broader trend in deep learning of treating what the model learned as itself a learnable object, analogous to Grad-CAM for CNNs or attention-rollout for Transformers.

References

[1] Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., and Crawford, K. Datasheets for datasets. *arXiv preprint arXiv:1803.09010*, 2018.

[2] Hamilton, W. L., Ying, R., and Leskovec, J. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017.

[3] Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[4] Luo, D., Cheng, W., Xu, D., Yu, W., Zong, B., Chen, H., and Zhang, X. Parameterized explainer for graph neural network. In *Advances in Neural Information Processing Systems*, 2020.

[5] Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T., and Leiserson, C. E. EvolveGCN: Evolving graph convolutional networks for dynamic graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(4):5363–5370, 2020.

[6] Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., and Bronstein, M. Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637*, 2020.

[7] Sankar, A., Wu, Y., Gou, L., Zhang, W., and Yang, H. DySAT: Deep neural representation learning on dynamic graphs via self-attention networks. In *Proceedings of the 13th ACM International Conference on Web Search and Data Mining*, pages 519–527, 2020.

[8] Weber, M., Domeniconi, G., Chen, J., Weidele, D. K. I., Bellei, C., Robinson, T., and Leiserson, C. E. Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591*, 2019.

[9] Yuan, H., Yu, H., Gui, S., and Ji, S. Explainability in graph neural networks: A taxonomic survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(5):5782–5799, 2022.

[10] Ying, R., Bourgeois, D., You, J., Zitnik, M., and Leskovec, J. GNNExplainer: Generating explanations for graph neural networks. In *Advances in Neural Information Processing Systems*, 2019.

[11] Zheng, L., Li, Z., Li, J., Li, Z., and Gao, J. AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, pages 4419–4425, 2019.